



LARGE LANGUAGE MODEL

Proxy-Pointer RAG: Achieving Vectorless Accuracy at Vector Scale and Cost

LATEST

EDITOR'S PICKS

A new way to build vector RAG— **Recommended Article** [Close](#)
reasoning-capable

Partha Sarkar

Apr 5, 2026 23 min read

WRITE FOR TDS

[Sign in](#)

[Submit an Article](#)



Generated using Gemini

The launch of PageIndex recently, is part of a broader architecture toward “**Vectorless RAG**” or “**Reasoning Retrieval**.” Instead of the standard method of splitting documents into random chunks and searching via similarity, PageIndex builds a “Smart Table of Contents (hierarchical tree) that allows LLMs to navigate doc

human expert would. Numerous blogs (including [this one](#) from Microsoft), outline the working principles (no vector database, no chunking, enhanced explainability) along with 98.7% accuracy achieved on a financial benchmark. However, they are also careful to note that Vectorless RAG is best suited for deep-dive queries on complex structured or semi-structured documents (such as financial statements), than searching across many independent documents, such as customer support knowledge we should continue to use Vector RAG.

LATEST

EDITOR'S PICKS

Why is that?

Recommended Articles

If Vectorless RAG using PageIndex provides better (or good) results on almost any query, why not use it for a collection of documents. The primary reason is that **tree-based approach cannot practically scale to many scenarios**. The hierarchical tree index that is a pre-ingestion step is slow and expensive to build using LLM. Furthermore, the retrieval is a 2 step process: use the tree, locate the most relevant nodes, then use those nodes as context for the response synthesis LLM.

In comparison, building a vector index is fast and ingestion and the retrieval step uses a LLM only once during the response synthesis step. Also, Ingestion using an embedding model costs less than summarization of the full document by an LLM.

What if you could get the excellent structure-aware accuracy of Vectorless RAG, along with the low latency of a Vector RAG, in a way that is scalable across thousands of documents in a database? In this article, I'll walk through a real use case of ingesting a large, complex document to build **Proxy-Pointer RAG**. I'll show the ingestion and retrieval pipeline that achieves this through several novel engineering steps. Along the way, we will experiment and demonstrate the following:

WRITE FOR TDS

Sign in

Submit an Article

- Why exactly is PageIndex so accurate? And why it is difficult to practically scale the concept to multi-document knowledge bases.
- A quick comparison of Vectorless RAG using PageIndex vs Flat Vector RAG to establish a baseline.
- How can we incorporate the principles of PageIndex into a Vector index with none of the associated latency and cost?
- Comparison of a wide variety of queries using [LATEST](#) Proxy-Pointer to test the quality of retrievals. [EDITOR'S PICKS](#)

Recommended Articles

Use Case Setup

We will use a World Bank report named [South Asia Update, April 2024: Jobs for Resilience](#) (License: CC BY-NC-SA). This is a 131 page report comprising multiple chapters, charts, tables, content in boxes etc. and is a good candidate for PageIndex to prove its capability. I have used `gemini-1.5-pro` the LLM to build the pageindex tree and `gemini-1.5-pro` for retrievals. I extracted the report pdf to a markdown using [Adobe PDF Extract API](#), but any other method such as `PyPDF2` or `VLM` which preserves the integrity of the tables, charts, and images will work just as well. For vector database, `FAISS` is used.

[WRITE FOR TDS](#)

[Sign in](#)

[Submit an Article](#)

How does PageIndex work?

Instead of the “*chunk your document, embed the chunks, store them in a vector database, retrieve the top-K, feed them to an LLM*” pipeline of a Vector RAG, PageIndex takes a radically different approach to document retrieval. Instead of treating a document as a flat set of chunks, it builds a **semantic skeleton tree** — a hierarchy of every section, sub-section, and content block in the document — and then uses an LLM to *navigate* that tree at c

Phase 1: Indexing(once per document)

PageIndex parses the document's heading structure (Markdown headers, PDF outlines, etc.) into a nested tree. Each node gets:

- A **title** (extracted from the heading)
- A **node ID** (unique identifier like 0012)
- **Line boundaries** (start and end line in the source document)
- A **summary** (generated by an LLM — this is the expensive and time-consuming part)

LATEST

EDITOR'S PICKS

The result is a JSON that looks like this:

Recommended Articles

```
{
  "node_id": "0011",
  "title": "Chapter 1. Deceptive Strength",
  "summary": "Covers South Asia's growth outlook",
  "line_num": 621,
  "nodes": [
    {
      "node_id": "0012",
      "title": "Introduction",
      "summary": "Summarizes the chapter's key the",
      "line_num": 625
    },
    ...
  ]
}
```

WRITE FOR TDS

Sign in

Submit an Article

Phase 2: Retrieval (Per Query)

When a user asks a question, PageIndex hands the *summaries* to an LLM and says, “**Which nodes can answer?**”. This is unlike a Vector RAG which relies on mathematical similarity between query and chunk to build the relevant context.

The LLM reads the summaries — not the full text — and returns a short list of node IDs. PageIndex then uses the li

to slice the *exact, contiguous, full* section from the original markdown file and passes it to the synthesis LLM.

Why this works so well?

PageIndex excels because of three architectural advantages:

1. Structural Navigation, Not Pattern Matching

When we ask “*What are the main messages of Chapter 10 on Growth and Vulnerabilities*,” PageIndex doesn’t search for chunks containing the words “growth” or “vulnerabilities.” Instead, it reads the summary of node 0011 (“Chapter 10: Growth and Vulnerabilities”) which says “Covers growth outlook, main vulnerabilities, climate risks, and policy challenges” and immediately knows this is the right node. It reasons about *relevance*, not *semantic and lexical similarity*.

LATEST

EDITOR’S PICKS

Recommended Articles

WRITE FOR TDS

Sign in

Submit an Article

2. Contiguous Context Extraction

Once the right nodes are identified, PageIndex **extracts an unbroken section** that the node represents, from the original Markdown — headers, sub-headers, bullet points, references, and all. The synthesis LLM receives context that reads like a properly authored document section, rather than a fragmented chunk with arbitrary boundaries.

3. Zero Chunk Boundary Artifacts

There are no overlapping chunks, no split sentences, no windows that start mid-paragraph. Every piece of context has a natural beginning (the section header) and a natural end (the section’s start). This dramatically reduces hallucinations from ambiguous context.

However, this strength comes at a cost—and this is where the approach starts to break down.

Why this is difficult to scale?

The reasons are **Cost and Latency** (ingestion as well as retrieval time). For our 131-page report, we get 137 structural nodes in the tree and therefore, PageIndex needs **137 LLM calls** just for the indexing phase — one summary per node. For a corpus of 50 documents, that’s ~7,000 LLM calls before we have used it to answer a single question.

Here is a comparative cost of PageIndex vs Vector

LATEST
EDITOR’S PICKS

Metric	PageIndex	Recommended Articles	
Indexing LLM calls	~137 per document	0	WRITE FOR TDS
Indexing time	5-10 minutes per document	30 seconds per document	Sign in
Indexing cost	\$\$\$ (LLM API fees)	\$ (embedding only)	Submit an Article
Multi-document support	Requires per-document tree traversal during retrieval	Single document fast access	
Retrieval quality	★★★★★	★★★	

For a single critical document such as a financial report, the cost is easily justified. For a 500-document enterprise knowledge base on customer service SOPs that updates weekly? It’s a tough starter.

This is where PageIndex starts becoming impracticable for large systems at scale.

Before proposing an alternative, let's establish a baseline: how much better is PageIndex compared to standard Vector RAG?

Comparison of Vectorless vs Flat Vector RAG

To establish a baseline, I compared the results of a diverse range of queries using Vectorless and flat Vector RAG. As expected, the quality of Vectorless RAG was better in all cases. Here are some examples:

LATEST

EDITOR'S PICKS

Query about table content

Recommended Articles

Here is the response from Vectorless RAG:

=== VECTORLESS RAG INTERACTION ===

Question: what does annexure table a2.1.1 say

Nodes Retrieved: 0125, 0131

Response:

Annex Table A2.1.1, titled "Studies included in the

The table catalogs 25 specific studies, detailing

- * **Paper:** The author(s) and publication source
- * **Period of main sample:** The timeframe covered
- * **Sample coverage:** The geographical or organizational
- * **Region:** The specific region analyzed.
- * **Economies:** The specific countries included

According to the provided text, the articles listed

And the same using Vector RAG, which completely misses the content

=== VECTOR RAG INTERACTION ===

Question: what does annexure table a2.1.1 say

Retrieved 5 chunks.

Response:

Based on the provided context, there is no information

The documents list "A2.1.1" in the table of contents

- * **Annex 2.1:** Describes a summary of the literature

WRITE FOR TDS

Sign in

Submit an Article

- * **Annex Figure A2.1.1 (referenced as A2..1.11.1 in the text):*
- * **Annex Table A2.3.3:** Provides specific regression data rega

There is no specific data or content provided for Annex Table A2.1

Query requiring Structural knowledge of the document

The query is what are the questions answered by chapter 2? Here is the response from Vectorless RAG. It precisely locates the section and the section of “Questions” within that and provides the questions verbatim, along with the grounding information for each question.

LATEST
EDITOR'S PICKS

Recommended Articles

=== VECTORLESS RAG INTERACTION ===

Question: what are the questions answered by chapter 2?
Nodes Retrieved: 0098

Response:

Based on the provided excerpts, Chapter 2 addresses the following questions:

- * **How does South Asia’s employment compare with other regions?
- * **What are the correlates of long-run employment growth?
- * **How can South Asia create the jobs needed to sustain growth?

These questions are explicitly listed in the section titled “Questions” within Chapter 2.

The chunks in Vector RAG do not maintain the structural hierarchy and section integrity, so while it locates the relevant section, it is not able to find the specific questions asked in the document.

=== VECTOR RAG INTERACTION ===

Question: what are the questions answered by chapter 2?
Retrieved 5 chunks.

Response:

Based on the provided document, Chapter 2—titled “Trends and Employment Ratios”—addresses the following questions:

The chapter explicitly identifies a “Questions” section within the chapter.

- * **Trends and Productivity:** The chapter examines the trends in productivity growth and its determinants.
- * **Employment Ratios:** A significant portion of the chapter is devoted to analyzing the employment ratios, including:
 - * **Policy and Structural Factors:** Tax regimes, labor market regulations, and structural changes in the economy.
 - * **Market and Financial Factors:** Access to credit, foreign investment, and market integration.
 - * **Human Capital and Firm Characteristics:** The role of education, skills, and firm-level characteristics in driving productivity and employment growth.

WRITE FOR TDS

Sign in

Submit an Article

- * **Growth Dynamics:** The chapter explores the "Engines of outp
- * **Policy Implications:** The research concludes by addressing

So the question becomes: **can we retain these advantages without paying the cost?**

Engineering a Better Retriever — Proxy-Pointer RAG

LATEST

Let’s build **Proxy-Pointer RAG** to answer that ques EDITOR’S PICKS
insight is intuitive and as follows:

Recommended Articles

You don’t need LLM summaries to give a vector ua structural awareness. You just need to encode the the embeddings themselves.

WRITE FOR TDS

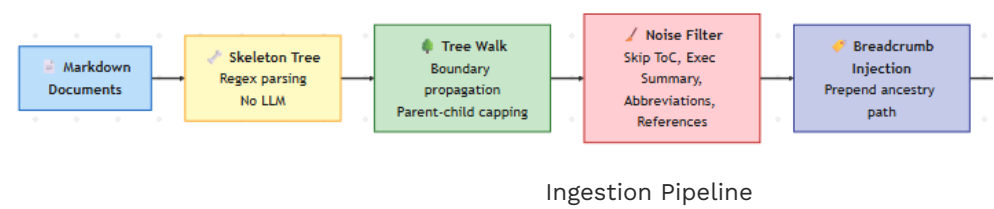
Sign in

Submit an Article

The system uses the same structural tree using Pa but *without the expensive summarization flag set*. skeletal tree requires no expensive LLM calls during tree is built purely from regex-based heading detec runs in milliseconds.

Then, instead of asking an LLM to navigate the tree do the retrieval — but we engineer the chunks so t “understands” where each chunk lives in the docu hierarchy.

Here is a view of the Ingestion pipeline:



Build a Skeleton Tree

PageIndex's tree parser doesn't actually *need* an LLM to build the structural hierarchy. The heading detection is regex-based — it finds Markdown headers (`#`, `##`, `###`) and builds the nesting from indentation levels. The LLM is only used to *summarize* each node.

We call the LLM-free version a **Skeleton Tree**: same structure, same node IDs, same line boundaries — but no summaries.

```
# Build skeleton tree – no LLM, runs in milliseconds
pageindex = PageIndex(doc_path, enable_ai=False)
tree = pageindex.build_structure() # Pure regex parser
```

LATEST
EDITOR'S PICKS

Recommended Articles

The skeleton tree and the summarized earlier Vectorless RAG produce **identical structure** nodes, same nesting depths, same line numbers, same only difference is the missing summary field.

WRITE FOR TDS

Sign in

Cost: \$0. Time: < 1 second.

Submit an Article

Structural Metadata Pointers (The Core Differentiator)

This is the heart of why PageIndex works so well — we will adopt.

In standard vector RAG, a retrieved chunk *is* the context. Whatever 500 words *FAISS* returns, that's what the the chunk starts mid-sentence or ends before the the response will miss the intent of the query entirely (illustrated in the previous section on Vectorless vs comparison).

PageIndex does something fundamentally different **never the context**. Each node in the tree knows its in the original document — its title, its node ID, and the **start and end line numbers of the entire section it represents**. When retrieval selects a node, PageIndex

to the original Markdown file and slices out the **full, contiguous section** between those line boundaries.

We replicate this exactly. Every chunk we embed into the vector index carries rich structural metadata from the tree node:

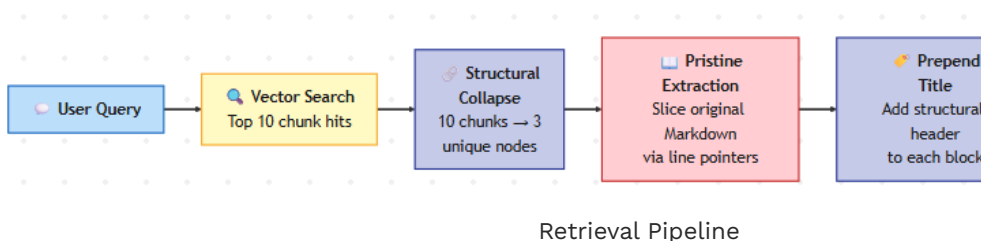
```
metadata = {  
    "doc_id": "SADU",           # Which document  
    "node_id": "0012",         # Which structural node  
    "title": "Introduction",    # Section heading LATEST  
    "start_line": 624,          # Where the section starts  
    "end_line": 672             # Where the section ends EDITOR'S PICKS  
}
```

Recommended Articles

At retrieval time, we don't feed the matched chunk
Instead, we:

1. **Use the chunks as proxies** — they're only there to identify *which nodes* are relevant. Remove duplicate (doc_id, node_id) combinations to get unique top-k.
2. **Follow the metadata pointers** — open the original slice lines of nodes, e.g. 624 to 672
3. **Deliver the full sections** — the LLM receives the pristine, structurally-intact text

Here is a view of the retrieval pipeline:



This means even if a chunk only matched on a single line deep inside a section, the synthesis LLM gets the *entire section* — with its header, its context, its figures, its references. The chunk was disposable; the pointer is what matters.

This is why I call it Proxy-Pointer RAG: the vectors are proxies for location, the metadata are pointers to the real content.

Cost: \$0. Impact: Transforms context quality from fragmented chunks to complete document sections.

Breadcrumb Injection (Structural Context)

This is key to answer the queries related to specific the document (such as Chapter 2). Standard vector raw text:

LATEST

EDITOR'S PICKS

Recommended Articles

"While private investment growth has slowed in both

FAISS has no idea this chunk comes from Chapter Economic Activity, inside Box 1.1. So when user asks *messages of Chapter 1*," this chunk won't rank high contain the words "Chapter 1" or "main messages."

WRITE FOR TDS

Sign in

Submit an Article

Breadcrumb injection prepends the full ancestry path Skeleton Tree to every chunk before embedding:

```
"[Chapter 1. Deceptive Strength > Economic activity]
While private investment growth has slowed in both
```

Now the embedding vector encodes *both* the content structural location. When someone asks about "Chapter 1" knows which chunks belong to Chapter 1 — because "Chapter 1. Deceptive Strength" are present in the

```
# Build breadcrumb from ancestry
current_crumb = f"{parent_breadcrumb} > {node_title}"

# Prepend to chunk text before embedding
enriched_text = f"[{current_crumb}]\n{section_text}"
chunks = text_splitter.split_text(enriched_text)
```

This is a **zero-cost encoding of the tree structure into the vector space**. We're using the same embeddings API, the same *FAISS* index, the same retrieval code. The only difference is what we feed into the embedder.

Cost: \$0 extra. Impact: Transforms retrieval quality for structural queries.

Structure Guided Chunking (No Blind Sliding Window) LATEST

EDITOR'S PICKS

Standard vector RAG applies a sliding window to the

document — a 2000-character window

Recommended Articles

some overlap, completely oblivious to the document structure.

A chunk might start mid-paragraph in the Introduction or end mid-sentence in a Figure caption. The boundaries are arbitrary and every chunk is an island in itself, with no knowledge of position in the overall document structure.

WRITE FOR TDS

Sign in

Submit an Article

Proxy-Pointer does something fundamentally different: it encodes **the tree, not the text**.

For each node in the skeleton tree, we extract *only section text* — from `start_line` to `end_line` — and pass that to the text splitter to that isolated section. If a section is short, it becomes a single chunk. If it's longer, the splitter splits it but strictly within that section's boundaries.

Standard **RAG**: Blind sliding window across entire document
[====chunk1====][====chunk2====][====chunk3====]..
↑ might start in Introduction, end in Figure caption

Proxy-Pointer: Chunk within each node's boundaries
Introduction (lines 624-672) → [chunk A] [chunk B]
Economic Activity (lines 672-676) → [chunk C]
BOX 1.1 (lines 746-749) → skipped (< 100 characters)
Inflation (lines 938-941) → [chunk D]

This guarantees three things:

1. **Chunks never cross section boundaries** — a chunk from Introduction will never overlap with Economic Activity
2. **Each chunk belongs to exactly one node** — so the `node_id` metadata is always precise
3. **Breadcrumbs are accurate per-chunk** — they reflect the actual structural container, not a guess

Importantly, when a node is skipped (because its title is empty — e.g., a “BOX 1.1” heading with no body content), the walk **still recurses into its children**. The actual parent-child nodes like “Introduction,” “Features,” and “Economic Activity” which get embedded with the parent’s title (eg: BOX 1.1 Accelerating Private Investment > Introduction > Features of...). No content is ever lost; only structural headers are excluded.

LATEST

EDITOR’S PICKS

Recommended Articles

WRITE FOR TDS

Sign in

Cost: \$0. Impact: Every chunk is structurally traceable, enabling precise metadata pointers.

Submit an Article

Noise Filtering (Removing Distractions)

Some sections in any document are **irrelevant for** a given query: they contain keywords that match almost every query but provide no useful content.

- **Table of Contents** — mentions every chapter title (“Chapter 1,” “Chapter 2,” everything)
- **Executive Summary** — paraphrases every key point (matches every topic query)
- **Abbreviations** — lists every acronym used in the document
- **Acknowledgments** — mentions organizations, people, themes

These sections act as **distractions** in vector space, pulling the centroid of the document’s embedding distribution away from the actual content.

retrieval toward shallow overview text instead of deep analytical content.

We apply a simple title-based filter during indexing:

```
NOISE_TITLES = {  
    "contents", "summary of contents", "table of contents",  
    "abbreviations", "acknowledgments", "foreword",  
    "executive summary", "references"  
}
```

```
if title.strip().lower() in NOISE_TITLES:  
    continue # Skip this node entirely
```

LATEST

EDITOR’S PICKS

Recommended Articles

This filter removed **7 nodes** from our test document. The impact was immediate: for the query “*What are the main points of Chapter 1?*”, the system stopped retrieving the Executive Summary and Table of Contents and started retrieving the Introduction section.

WRITE FOR TDS

Sign in

Submit an Article

Cost: \$0. Impact: Eliminates the most common retrieval mode.

That’s it. Proxy-Pointer RAG is now ready to test.

Test Proxy-Pointer pipeline

While detailed comparison is mentioned in the next section, here are the results of the same two queries that were discussed in the “Vectorless vs Flat Vector RAG” section. As you can see, the answers are as accurate and detailed as Vectorless RAG.
PageIndex.

Query about table content

=== PROXY-POINTER RAG INTERACTION ===

Question: what does annexure table a2.1.1 say

Unique Context Nodes Extracted:

- [SADU] Node 0131: 'ANNEX TABLE A2.1.1 Studies

- [SADU] Node 0092: 'ANNEX TABLE SL.B1.1.1 Marginal probability
- [SADU] Node 0052: 'TABLE B1.1.1 Correlates of the likelihood o

Response:

Based on the provided documents, **Annex Table A2.1.1** is titled

This table provides a summary of various research studies, detaili

- * **Paper:** The author(s) and year of the study (e.g., Kapsos (
- * **Period of main sample:** The timeframe covered by the study
- * **Sample coverage:** The scale or number of entities analyzed
- * **Region:** The geographic scope of the research
- * **Economies:** The specific countries or econo

LATEST

EDITOR'S PICKS

The table documents a diverse set of r

Recommended Articles

Query requiring Structural knowledge

=== PROXY-POINTER RAG INTERACTION ===

Question: what are the questions answered by chapt

Unique Context Nodes **Extracted:**

- [SADU] Node 0099: 'FIGURE 2.3 Growth models' |
- [SADU] Node 0098: 'Questions' | **Lines:** 3115-31
- [SADU] Node 0124: 'ANNEX 2.1 Summary of litera

Response:

Based on the provided text, Chapter 2 addresses th

- * **How does South Asia's employment compare wit**
- * **What are the correlates of long-run employe**
- * **How can South Asia create the jobs needed to**

Additionally, the chapter explores these issues wi

While we have demonstrated the cost advantage s
architecture overcome the scalability challenge of

Is Proxy-Pointer Scalable?

Here is why the architecture is scalable across an
knowledge base. PageIndex pays a scalability pena
ends: ~137 LLM calls per document during indexing

WRITE FOR TDS

Sign in

Submit an Article

additional LLM reasoning step per query for tree navigation.

Proxy-Pointer eliminates both.

- **No LLM at indexing.** The skeleton tree is regex-built in milliseconds. The only API calls are to the embedding model — identical to standard vector RAG.
- **No tree navigation at retrieval.** Queries go straight to the vector index. No LLM reading summaries, no p

LATEST

EDITOR'S PICKS

Proxy-Pointer is standard vector RAG

baked in. The structural awareness liv

Recommended Articles

(via breadcrumbs) and the chunk metadata (via no not in an LLM reasoning loop. It inherits all of vect scalability: unified multi-document indexes, sub-li incremental updates, and zero per-query LLM over the final synthesis.

WRITE FOR TDS

Sign in

Submit an Article

Fail-safe for unstructured documents: If a docume headings — or the skeleton tree produces only a si — the system detects this during chunking and fal standard sliding window. Chunks are flagged with empty `node_id` and line boundaries. At retrieval tim chunks are used directly as LLM context instead o pointers back to the source. The system gracefully standard vector RAG — no errors, no special handl

Let's compare Vectorless RAG and Proxy-Pointer h

Vectorless vs Proxy-Pointer RAG

I ran a variety of queries — broad structural, cross specific factual, figure specific etc. And let Claude responses for a comprehensive comparison. You ca

detailed responses from Vectorless and Proxy-Pointer along with the full [Quality Comparison report here](#).

The following table encapsulates the verdict. The **Final Score: PageIndex 2 — Proxy 4 — Ties 4 . In other words, Proxy-Pointer matches or beats PageIndex on 8 out of 10 queries. And all at the scalability and price of a Flat Vector RAG.**

Here is the summary verdict:

			LATEST
			EDITOR'S PICKS
#	Query Type	Recommended Articles	
1	Broad structural (Ch.1 messages)	Paq	
2	Broad structural (Ch.2 messages)	Paq	
3	Specific factual (Box 1.1 features)	Tie	
4	Cross-reference (inflation tables)	Pro	
5	Comparative (India vs region)	Pro	
6	Figure-specific (B1.1.1 trends)	Pro	
7	Direct lookup (Annexure A2.1.1)	Tie	
8	Entity-specific (currency crisis countries)	Tie	

WRITE FOR TDS

Sign in

Submit an Article

#	Query Type	Winner
9	Navigational (Ch.2 questions)	 Tie
10	Inferential/policy (govt vs shocks)	 Proxy-Pointer

And here is the cost comparison:

LATEST

EDITOR'S PICKS

Metric	PageIndex	Prox	Recommended Articles
Indexing LLM calls	~137 per doc	0	WRITE FOR TDS
Indexing time	5-10 min/doc	< 30 sec/doc	Sign in
Retrieval quality	★★★★★	★★★★★ (8/10 vs PageIndex)	Submit an Article
Multi-doc scalability	Poor (per-doc tree nav)	Excellent (unified vector index)	
Structural awareness	Full (LLM-navigated)	High (breadcrumb-encoded)	
Index rebuild on update	Expensive (re-summarize)	Cheap (re-embed affected nodes)	
Explainability	High (section titles + doc IDs)	High (section titles + doc IDs)	

Key Takeaways

1. **Structure is the missing ingredient in RAG.** The quality gap between naive vector RAG and PageIndex isn't about better embeddings — it's about preserving hierarchy.
2. **You don't need an LLM to encode structure.** Breadcrumb injection and structural metadata give the vector index, structural awareness without any cost.
3. **Noise filtering beats better embeddings.** Removing low-value nodes from the index had more impact on retrieval quality than any model swap could.
4. **Pointers beat chunks.** Chunks act as proxies in a hierarchical section, which is what the synthesizer LLM sees.

LATEST

EDITOR'S PICKS

Recommended Articles

WRITE FOR TDS

Sign in

Submit an Article

Conclusion

Proxy-Pointer RAG proves a simple thesis: **you don't need an expensive LLM to make a retriever structurally aware. You need to be clever about what you embed.**

Five zero-cost engineering techniques — skeleton metadata pointers, breadcrumbs, structure-guided noise filtering — close the quality gap with a full LLM system, while keeping the speed and scalability of vector RAG. On our 10-query benchmark, Proxy-Pointer RAG or beat PageIndex on 8 out of 10 queries, at the cost of standard Vector RAG.

The next time you're building RAG for your structured (or unstructured) document repository, don't reach for a better model. Reach for **Proxy-Pointer** index.

Connect with me and share your comments at www.linkedin.com/in/partha-sarkar-lets-talk-AI

Reference

World Bank. 2024. South Asia Development Update, April 2024: Jobs for Resilience — License: CC BY 3.0 IGO.

Images used in this article are generated using Google Gemini. Code created by me.

LATEST

EDITOR’S PICKS



Recommended Articles

WRITTEN BY

Partha Sarkar

See all from Partha Sarkar

WRITE FOR TDS

Sign in

Submit an Article

Ai Engineering

Artificial Intelligence

Deep Dives

Rag Architecture

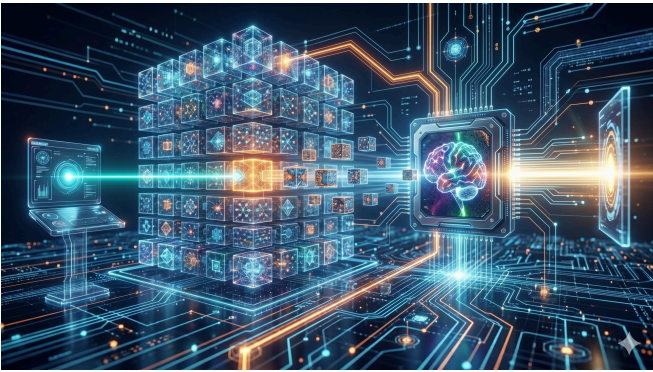
Share This Article



**Towards Data Science is a community publica
your insights to reach our global audience and
the TDS Author Payment Program**

Write for TDS

Related Articles



LARGE LANGUAGE MODEL

Proxy-Pointer RAG: Structure Meets Scale at 100% Accuracy with Smarter Retrieval

Open source. 5-minute setup. Vector RAG done right—try it yourself.

Partha Sarkar

April 19, 2026 14 min read

LATEST

EDITOR'S PICKS

Recommended Articles

WRITE FOR TDS

Sign in

Submit an Article



Subscribe 1

Your home for data science and AI. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

WRITE FOR TDS ·

PRIVACY POLIC